

UNIVERSITY OF SOUTHERN CALIFORNIA

Statistical genetics on All of Us

Lab 1: a phenotype, a polygenic index, and a regression on v9 data

Jeffrey M. Swigert, PhD

Center for Economic and Social Research

September 2026

What we will do today

Step	What we do	Script (in scripts/)
0	Start a cloud environment, get the code, fill in settings	00_setup.sh
1	Put chromosome-22 genotypes and ancestry data on the VM	01_fetch_genotypes.sh
2	Build a height phenotype from the CDR	02_build_phenotype.sh
3	Summarize it: a summary table and two figures	03_explore_phenotype.sh
4	Build a polygenic index (PGI); check its histogram	04_build_pgi.sh
5	Regress height on the PGI and covariates	05_pgi_regression.sh
6	Save the run; download aggregate results	06_save_run.sh

For each step: a slide on what it does, then a  **Run it** slide with the commands and the code.

Every command is also in `COMMANDS.txt` in the repository, with a note on each, to copy and paste from.

All of Us data use rules

Rule	In this lab
Participant-level data stays in the Workbench: no copying, downloading, or screenshots.	Person-level files live in <code>lab_data/</code> and <code>work/</code> only.
No count of 1–20 participants may be shared, directly or by calculation.	Step 6 checks <code>results/</code> before anything is saved or downloaded.
Downloads are monitored; downloading participant-level data triggers an alert.	Downloads come from <code>results/</code> only.
No participant data or counts under 20 in code on GitHub.	<code>.gitignore</code> excludes data and output folders.
Notify the program 2 weeks before a publication or presentation.	For your own projects.

Setting up your cloud environment

In the workshop workspace: **Apps** tab → + **New app instance** → **JupyterLab**.

Setting	Choose	Why
Environment	Standard VM	The Spark cluster option is for Hail.
Machine	n1-standard-8: 8 CPUs, 30 GB memory	The first machine in the menu. Enough for one chromosome.
Disk	100 GB	Today uses under 5 GB. Disks grow, never shrink.
GPU	None	Nothing today uses a GPU.
Autostop	On, 1 hour	Stops the VM after 1 idle hour: a clean stop when the lab is over.

The creation page shows the cost of the choices: about \$0.41 per hour for this machine. A running VM bills by the hour; a stopped VM bills only for its disk.

Autostop: when to change it

Autostop stops the VM when it has been idle. The VM counts as **idle** when, for 15 minutes,

- average CPU use is below 50%, and
- no browser tab or terminal connection is active.

Keep autostop on (1 hour today) for interactive work.

Set it longer than the job (up to 14 days) when a job runs for hours with low CPU use:

- a single-threaded program on an 8-CPU VM (1 busy CPU = 12.5% use);
- a large copy from a bucket, which waits on the network, not the CPU;
- a script that waits on a long BigQuery query.

Otherwise the VM can stop in the middle of the job. **Turn autostop off** only if you will stop the VM yourself.

GitHub: what it is and how we use it

Git records versions of the files in a folder. Each saved version is a *commit*.

GitHub hosts a copy of that folder (a *repository*) online, so others can copy it.

We use it for three things:

1. **Getting the code onto the VM:** `git clone` once, `git pull` for updates.
2. **Recording which code produced a result:** step 6 writes the commit ID into `results/run_manifest.txt`.
3. **Running identical code** in every participant's workspace.

GitHub is outside the Workbench. A repository holds code, settings templates, and documentation. It should never hold data, results, filled-in settings, or notebook outputs.

.gitignore: keeping data out of GitHub

A .gitignore file lists files and folders that git never tracks. This repository's:

```
config.sh          # your filled-in settings
lab_data/          # copied-in data (person-level)
work/              # intermediate files (person-level)
results/           # aggregate outputs (reviewed and downloaded by hand)
runs/              # saved archives
.ipynb_checkpoints/ # notebook autosaves
```

Practices that carry over to any project:

- Put data and outputs in folders, and ignore the folders by name.
- Commit a settings template (config.example.sh); ignore the filled-in copy (config.sh).
- Clear notebook outputs before committing (JupyterLab: Edit → Clear All Outputs). A saved .ipynb file stores its outputs.
- Before git push, run git status and git diff -cached to see exactly what will be sent.

■ Run it – Step 0: get the code and fill in settings

JupyterLab Launcher → **Terminal**, then (or paste from COMMANDS.txt):

```
git clone https://github.com/jeffreyswigert/aou-statgen-lab.git # copy the repository here
cd aou-statgen-lab # move into it
cp config.example.sh config.sh # your own copy of the settings
nano config.sh # fill in two values; Ctrl-O saves, Ctrl-X exits
git check-ignore -v config.sh # prints the .gitignore line that excludes config.sh
bash scripts/00_setup.sh # check tools, settings, disk
```

The two values to fill in (shown on screen):

```
export WORKSHOP_BUCKET="gs://..." # the workshop workspace's bucket
export BILLING_PROJECT="..." # the Google project billed for queries and copies
```

Expected: the last line reads Setup check passed. If a value is missing, the script names it and stops.

Code: scripts/00_setup.sh; scripts/common.sh (settings loaded by every script)

Where today's data are

1. The CDR, in BigQuery: `wb-silky-artichoke-2408.C2025Q4R6`

Tables `measurement`, `person`, `concept` → the height phenotype (step 2)

2. The All of Us controlled bucket: `gs://vwb-aou-datasets-controlled/v9/`

`wgs/short_read/snpindel/aux/ancestry/ancestry_preds.tsv`: predicted ancestry and 16 PCs per person (step 1)

`wgs/short_read/snpindel/acaf_threshold/`: whole-genome genotypes, one file set per chromosome (next slide)

3. The workshop bucket: `$WORKSHOP_BUCKET`

`genotypes/hm3_hg38.tsv`: the public HapMap3 variant list (positions and alleles, no people)

`genotypes/chr22_hm3.{bed,bim,fam}`: the chromosome-22 subset, ready made (step 1)

`pgi_workshop/height_weights.txt`: PGI weights from a published height GWAS (step 4)

The controlled bucket is **requester-pays**: each copy names who pays (`-billing-project`). PLINK reads files on the VM's disk, so step 1 puts them there.

Where the genotypes come from

- All of Us publishes whole-genome genotypes (the ACAF callset) as one file set per chromosome, in PLINK 2 and PLINK 1 formats. The chromosome-22 .bed is about 250 GB. There is no HapMap3 subset.
- **HapMap3 variants:** about 1.2 million common variants from the International HapMap Project. Most GWAS report them, so they are a standard set for building PGIs.
- So a PGI project starts by keeping those variants and renaming them to rsIDs. `scripts/subset_aou_genotypes.sh` does it with two PLINK commands:

```
# keep the HapMap3 variants and the chosen people; name variants chr22:pos:ref:alt
plink2 --pfile acaf --set-all-var-ids 'chr@:#:$r:$a' \
      --extract extract.txt --keep keep.txt --make-bed --out hm3_aouids
# rename the variants to rsIDs, to match the GWAS weights
plink2 --bfile hm3_aouids --update-name rename.txt --make-bed --out chr22_hm3
```

`extract.txt`, `rename.txt`, and `keep.txt` come from matching the HapMap3 list against All of Us's variant file by position and alleles (`scripts/make_hm3_lists.py`).

■ Run it – Step 1: the genotypes and the ancestry file

```
bash scripts/01_fetch_genotypes.sh      # 1-3 minutes; start it, then read on
```

What the script does, line by line (`scripts/01_fetch_genotypes.sh`):

Line	What it does	In your own project
<code>source scripts/common.sh</code>	loads <code>config.sh</code> , the safety switches, the billing flag	one shared setup file
<code>mkdir -p lab_data</code>	a folder for copied-in data	data outside the code folder
<code>for ext in bed bim fam; do ... done</code>	runs the copy three times, once per extension	files that travel together
<code>gcloud ... storage cp SRC DEST</code>	copies one file from a bucket to disk	the same command for any <code>gs://</code> file
<code>gcloud ... cp "\$ANC_SRC" lab_data/...</code>	the ancestry file, from the requester-pays bucket	<code>-billing-project</code> names who pays
<code>ls -lh ...; wc -l < FILE</code>	file sizes; lines per file	check every copy

■ Run it – Step 1: look at what arrived

When the script finishes, look at the first lines of each file:

```
head -n 2 lab_data/chr22_hm3.fam          # first 2 lines: one line per person
head -n 2 lab_data/chr22_hm3.bim          # one line per variant
head -n 1 lab_data/ancestry_preds.tsv | tr '\t' '\n' # the header, one column name per line
```

Piece	Meaning
<code>head -n N FILE</code>	print the first N lines of FILE (the way to look at any text file)
<code>A B</code>	a <i>pipe</i> : what A prints goes into B instead of onto the screen
<code>tr '\t' '\n'</code>	replace every tab with a line break, so a wide header reads as a list

Expected: the script's last lines read chr22: ... variants staged, about 17,000 variants. Everyone's count is the same.

The files we just copied

File (one line =)	Columns	Note
chr22_hm3.fam (person)	FID, IID, father, mother, sex, phenotype	FID is 0 for everyone. IID is the person ID.
chr22_hm3.bim (variant)	chromosome, ID, cM, position, allele 1, allele 2	ID is an rsID; positions are GRCh38.
chr22_hm3.bed	genotypes, binary	Never open as text.
ancestry_preds.tsv (person)	research_id, ancestry_pred, probabilities, pca_features, ancestry_pred_other	research_id is the CDR person_id. pca_features holds 16 PCs in one bracketed text field.

Made-up lines. One person ID, three column names: IID, person_id, research_id.

chr22_hm3.fam	chr22_hm3.bim	ancestry_preds.tsv
0 1000010 0 0 2 -9	22 rs1234567 0 17054321 A G	1000010 eur [0.01,...] [-0.009,...] eur

Where phenotype data live in All of Us

Source	What it holds	CDR table(s), and how rows are labeled
Program measurements	Height, weight, BMI, waist and hip, blood pressure, heart rate; taken at enrollment, some self-reported remotely	measurement; measurement_source_concept_id in the All of Us (PPI) vocabulary, e.g. 903133
Surveys	The Basics, Overall Health, Lifestyle, health history, health care access, social determinants of health, and more	observation (also ds_survey); question = observation_source_concept_id, answer = value_source_concept_id
Electronic health records	Diagnoses, medications, procedures, lab results and vitals, visits; from participating sites	condition_occurrence, drug_exposure, procedure_occurrence, measurement, visit_occurrence; standard OMOP concepts, e.g. height 3036277
Wearables (Fitbit)	Daily activity, heart rate, sleep	activity_summary, heart_rate_summary, sleep_daily_summary, ...; one row per person-day or minute
Demographics	Year of birth, sex at birth, race, ethnicity	person

One variable can come by more than one route: height is in measurement twice, measured by the program (903133) and copied from health records (3036277).

The Data Browser: finding a variable before you query

`databrowser.researchallofus.org`: public, no login; participant counts rounded to the nearest 20.

Question	Where	What you get
Which survey asks about X, and how was it worded?	Surveys	the question text, answer options, participants per answer, concept IDs
How many participants have a diagnosis, drug, lab, or procedure?	EHR domains	concept name and ID, participant count; for labs, the value distribution
What does a program measurement look like?	Physical Measurements	distributions of height, weight, blood pressure, . . . by sex and age
Is a variant in the data?	Genomic Variants	search by gene, rsID, or position; allele counts and frequencies

The concept ID on each card goes in a query, or in a row of `data/phenotypes.tsv`. The same lookup in SQL:

```
SELECT concept_id, concept_name FROM '$CDR_DATASET.concept'  
WHERE LOWER(concept_name) LIKE '%height%'
```

Height: the concept choice

- Height is in the CDR measurement table: one row per measurement, often several rows per person.
- We select rows with `measurement_source_concept_id = 903133`: height measured by the program at enrollment, plus remote self-report.
- We do not select on the standard concept 3036277: it also returns heights from electronic health records.
- Year of birth and sex at birth come from the person table.
- `SAMPLE_MOD=1`, today's setting, keeps everyone. `SAMPLE_MOD=10` keeps people whose `person_id` is divisible by 10, about 1 in 10: a quick trial run when you change the code.

The query and the files it writes

The first of two queries (scripts/02_build_phenotype.sh):

```
SELECT person_id, value_as_number      -- which columns
FROM ' $CDR_DATASET.measurement'      -- which table
WHERE measurement_source_concept_id IN (903133)  -- which rows: the concept,
      AND value_as_number IS NOT NULL          -- with a value,
      AND MOD(person_id, 1) = 0               -- everyone (10: 1 in 10)
```

The second gets year_of_birth and sex_at_birth from person. Results go to work/ (made-up values):

```
work/raw_pheno.csv
person_id,value_as_number
1000010,172.4
1000010,172.0
1000020,158.9
```

```
work/raw_person.csv
person_id,year_of_birth,sex_at_birth
1000010,1961,Female
1000020,1983,Female
```

Note the two rows for person 1000010 in raw_pheno.csv: the next step reduces them to one.

■ Run it – Step 2: build the phenotype

```
DRY_RUN=1 bash scripts/02_build_phenotype.sh    # prints the two queries; runs nothing
bash scripts/02_build_phenotype.sh              # runs the queries, then cleans (1-2 min)
```

The cleaning (scripts/build_pheno.py, steps 1–5, lightly simplified):

```
value = statistics.median(values)                # 1: each person's median
if value < L0 or value > HI: continue              # 2: drop outside 100-250 cm
if sex not in ('Male', 'Female'): sex = 'other'    # 3: sex at birth
age = 2026 - int(yob)                             # 4: age
z = (value - mu) / sd                             # 5: standardize within sex
```

Output work/aou_pheno.tsv, one row per person: person_id value value_z age sex.

Expected (the script prints the sample at each step):

```
measurement records pulled    ...
people with any measurement    ...
removed: outside 100-250 cm   ...
people in analysis file       ...
```

Cleaning and exploratory data analysis: the summary table

Input: work/aou_pheno.tsv. Made-up rows, showing the columns:

person_id	value	value_z	age	sex
1000010	172.2	1.3172	65	Female
1000020	158.9	-0.4127	43	Female

Check it from the terminal (output stays on your screen):

```
head -n 3 work/aou_pheno.tsv | column -t      # first 3 lines, columns lined up
wc -l work/aou_pheno.tsv                      # count lines: rows + 1 header line
cut -f 5 work/aou_pheno.tsv | sort | uniq -c   # keep column 5, sort it, count each value
```

Outputs in results/: aou_pheno_summary.txt (the table), aou_pheno_hist.png, aou_pheno_kde_sex.png.

The table has one row per group with Obs, Mean, Std. dev., and p25 / Median / p75. Percentiles stand in for Min and Max, because each of those is one person's value.

Code: scripts/pheno_eda.py - summary_row() makes each table row

■ Run it – Step 3: the summary table and figures

```
bash scripts/03_explore_phenotype.sh
cat results/aou_pheno_summary.txt
```

Expected layout (made-up numbers):

Variable	Obs	Mean	Std. dev.	p25	Median	p75
height	...	169.10	9.80	162.0	168.9	176.2
Male	...	175.90	7.40	171.0	175.9	180.8
Female	...	162.40	6.90	157.8	162.3	167.0

Check: US adult means are about 176 cm (men) and 162 cm (women); SDs 7–8 cm.

Open the figures by double-clicking them in the JupyterLab file browser, folder `results/`.

Building the PGI

For person i , the PGI adds up, over variants j , the variant's weight times the person's number of copies of the effect allele:

$$\text{PGI}_i = \sum_j w_j g_{ij}, \quad g_{ij} \in \{0, 1, 2\}$$

The weights w_j are per-allele effect estimates from a published height GWAS (genome-wide association study).

File	Contents
input lab_data/chr22_hm3.{bed,bim,fam}	genotypes
input lab_data/height_weights.txt	weights: rsid effect_allele weight
output work/aou_pgi.sscore	one row per person; SCORE1_SUM is the PGI
output work/aou_pgi.log	PLINK's log, including the match count

Where a genotype is missing, PLINK uses the variant's average allele count.

The PLINK command

From scripts/04_build_pgi.sh (with CHROM=22):

```
p2 --bfile lab_data/chr22_hm3 \  
  --score lab_data/height_weights.txt 1 2 3 header cols=+scoresums list-variants \  
  --out work/aou_pgi
```

Part	Meaning
p2	plink2 with our CPU and memory settings (scripts/common.sh)
-bfile lab_data/chr22_hm3	read chr22_hm3.bed, .bim, .fam
-score FILE 1 2 3	weights file; variant ID in column 1, effect allele in 2, weight in 3
header	the first line of the weights file is column names
cols=+scoresums	also write each person's sum (SCORE1_SUM)
list-variants	write the variants used to work/aou_pgi.sscore.vars
-out work/aou_pgi	prefix for the output files

■ Run it – Step 4: build the PGI

```
bash scripts/04_build_pgi.sh           # weights, PLINK, QC histogram
grep -E 'variants processed|skipped' work/aou_pgi.log # the match count, from the log
wc -l < lab_data/height_weights.txt     # weight rows + 1 header line
```

The match count. `variants processed` is the number of weight rows that found their variant in the genotype file. PLINK skips unmatched rows without an error.

- Processed close to the number of weight rows: expected.
- Processed near zero: the variant IDs do not match (for example, rsIDs in the weights and `chr22:16050075:A:G` in the `.bim`), or the column numbers are wrong.

grep prints the lines of a file that match a pattern; `-E` allows the `|` (“or”) in the pattern.

QC on the PGI: a histogram

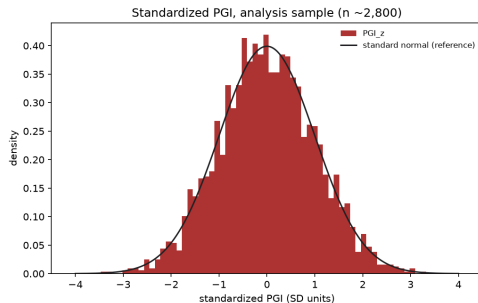
Step 4 standardizes the PGI and plots its histogram with a normal curve for reference:

$$\text{PGI_}z_i = \frac{\text{PGI}_i - \text{mean}(\text{PGI})}{\text{SD}(\text{PGI})}$$

```
pgi_z = (raw - raw.mean()) / raw.std()  
ax.hist(pgi_z, bins=60, density=True)
```

Expected: one hump centered at 0, close to the curve; skewness near 0 and kurtosis near 3 (in results/aou_pgi_summary.txt).

Code: scripts/pgi_eda.py



Example output of this code on simulated data (not All of Us data).

The regression

$$\text{height_}z_i = \alpha + \beta \text{PGI_}z_i + \gamma \text{age}_i + \delta \text{female}_i + \sum_{k=1}^5 \theta_k \text{PC}_{ik} + \varepsilon_i$$

Variable	Definition	File (step)
height_z	(height – mean for that sex) / SD for that sex	work/aou_pheno.tsv (2)
PGI_z	(PGI – mean) / SD, over the people in the model	work/aou_pgi.sscore (4)
age	2026 – year of birth	work/aou_pheno.tsv (2)
female	1 if Female, 0 if Male	work/aou_pheno.tsv (2)
PC ₁ –PC ₅	first 5 genetic principal components	lab_data/ancestry_preds.tsv (1)

Estimated by ordinary least squares (OLS).

What the regression reports

- β : height and the PGI are both in SD units, so β is the difference in height, in SDs, per 1 SD of PGI, with age, sex, and the PCs held fixed.
- **The PCs** summarize broad genetic ancestry. Including them keeps ancestry-related differences in height from being attributed to the PGI.
- **Incremental** $R^2 = R^2$ of the full model $- R^2$ of the same model without PGI_z. It is the share of height variance the PGI explains beyond the covariates.
- **Scope of today's estimate**: one chromosome, and OLS, which treats participants as unrelated. A full PGI uses all 22 chromosomes.

■ Run it – Step 5: the regression

```
bash scripts/05_pgi_regression.sh
cat results/aou_pgi_regression.txt
```

The fit (scripts/pgi_regression.py):

```
X[:, 0] = (X[:, 0] - X[:, 0].mean()) / X[:, 0].std() # PGI -> PGI_z
beta, se, r2_full = fit(X) # height_z on PGI_z, age, female, PC1-PC5
_, _, r2_cov = fit(X[:, 1:]) # the same model without PGI_z
```

Expected layout (made-up numbers):

term	estimate	SE	t
PGI_z	0.0XXX	0.00XX	X.XX
...			
R2 with PGI	= 0.0XXX		
R2 covariates	= 0.0XXX		
incremental R2	= 0.00XX		

After class: delete the five PC names from COVARs in scripts/pgi_regression.py, re-run step 5, and compare the PGI_z row.

Saving results and downloading them

Folder	Contents	Can it leave the Workbench?
lab_data/	copied-in genotypes, ancestry file, weights	No
work/	person-level files: phenotype, PGI scores	No
results/	summary tables (.txt), figures (.png)	Yes, after the check and your review
runs/	archive of the run (copied to the bucket)	No

To download a file: open it and read it; then JupyterLab file browser → right-click → **Download**.

- Figures and free text are not checked by the script; review them yourself.
- Copying out with `gcloud storage cp` to storage outside the Workbench is blocked.
- A large summary file needs an approved exception first (the large-download request process).

What step 6 does

`scripts/06_save_run.sh`:

1. **Checks** `results/` (`scripts/check_disclosure.py`) for counts of 1–20, direct or from the difference of two counts, and for person-level files. If it finds any, it stops and names the file.
2. **Writes** `results/run_manifest.txt`: the git commit ID, the settings, the PLINK version, and checksums of the input files.
3. **Packs** `config.sh`, `scripts/`, `work/`, and `results/` into one archive in `runs/`.
4. **Copies** the archive to `$WORKSHOP_BUCKET/statgen-lab/` and lists it to confirm.

The archive holds person-level files, so it stays in the workspace bucket. It is never downloaded.

■ Run it – Step 6: save, download, stop

```
bash scripts/06_save_run.sh
cat results/disclosure_report.txt           # what the check found
grep code_version results/run_manifest.txt  # the commit that produced this run
gcloud storage ls "$WORKSHOP_BUCKET/statgen-lab/" # list the bucket folder: the archive is
there
```

Expected: none found twice, then the archive's bucket path.

If the check stops the save: it prints the file and the count. Round, combine, or remove that cell, then re-run step 6.

Download one file: file browser → results/ → right-click aou_pgi_hist.png → Download.

Stop the VM: Apps tab → your JupyterLab app → **Stop**. Stop keeps the app and its disk: your files are there next time. (Autostop would do it for you after an idle hour.)

Code: `scripts/06_save_run.sh`; `scripts/check_disclosure.py`

All the commands, in order

```
git clone https://github.com/jeffreyswigert/aou-statgen-lab.git && cd aou-statgen-lab
cp config.example.sh config.sh && nano config.sh  # WORKSHOP_BUCKET, BILLING_PROJECT
bash scripts/00_setup.sh                          # check tools and settings
bash scripts/01_fetch_genotypes.sh                 # genotypes + ancestry file      -> lab_data/
bash scripts/02_build_phenotype.sh                 # CDR queries, one row per person -> work/
bash scripts/03_explore_phenotype.sh               # summary table, 2 figures          -> results/
bash scripts/04_build_pgi.sh                       # PGI, match count, histogram      -> work/, results/
bash scripts/05_pgi_regression.sh                  # the regression                    -> results/
bash scripts/06_save_run.sh                        # check, manifest, archive to the workspace bucket
```

Settings to change (in config.sh, or before one command):

- PHENO=ld1 – another phenotype (bash scripts/02_build_phenotype.sh list)
- SAMPLE_MOD=10 – 1 person in 10 instead of everyone (a quick trial run)
- GENO_SOURCE=aou – build the subset from All of Us's files; CHROM=21 – another chromosome

Also in the repository: COMMANDS.txt and cheatsheet.pdf.